

Python: warm-up exercises on iteration

This notebook was generated in **pseudocode** mode.

Exercise: Coin Flipping Simulation

Exercise Description

Write a piece of code that simulates coin tosses and keeps track of heads and tails.

Requirements:

- Use a variable `n` to specify the number of coin tosses
- Keep track of the number of heads and tails
- Use `random.randint(1, 100)` to simulate coin tosses (>50 = heads, ≤ 50 = tails)
- Store the final counts in variables `heads` and `tails`
- Return the variables `heads` and `tails`

Your solution

```
import random

def simulate_coin_flips(n):
    # step 1: initialize heads and tails counters to 0
    # step 2: loop from 0 to n-1
    # step 3: generate a random int in [1, 100]
    # step 4: if value > 50 increment heads else increment tails
    # step 5: return heads, tails
```

Test your solution

```
# Test that heads + tails equals total number of tosses
n = 10
heads, tails = simulate_coin_flips(n)
total_tosses = heads + tails
assert total_tosses == n, f"Expected {n} total tosses, got {total_tosses}"
assert heads >= 0 and tails >= 0, 'Counts should be non-negative'
print('Test passed: coin flipping simulation works correctly!')
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Sum of Number Sequence

Exercise Description

Write a piece of code that calculates the sum of a sequence of numbers.

Requirements:

- Keep track of the sum of numbers processed so far
- Stop when you encounter 0 (don't include 0 in the sum)
- Store the final sum in a variable called `total_sum`
- Return `total_sum`

Your solution

```
def sum_until_zero(numbers):  
    # step 1: initialize total_sum to 0  
    # step 2: iterate numbers in order  
    # step 3: if number is 0, break the loop  
    # step 4: otherwise add number to total_sum  
    # step 5: return total_sum
```

Test your solution

```
numbers = [5, 3, 8, 2, 0]  
total_sum = sum_until_zero(numbers)  
expected_sum = 18  
assert total_sum == expected_sum, f"Expected sum {expected_sum}, got {total_sum}"  
print('Test passed: sum calculation works correctly!')
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Count Odd and Even Numbers

Exercise Description

Write a piece of code that counts the number of odd and even numbers in a given range.

Requirements:

- Count numbers in the range from 1 to n (inclusive)
- Count how many numbers are odd and how many are even
- Store the counts in variables `odd_count` and `even_count`
- Return the variables `odd_count` and `even_count`

Your solution

```
def count_odd_even(n):  
    # step 1: initialize odd_count and even_count to 0  
    # step 2: loop i from 1 to n inclusive  
    # step 3: if i % 2 == 0 increment even_count else increment odd_count  
    # step 4: return odd_count, even_count
```

Test your solution

```
n = 10
odd_count, even_count = count_odd_even(n)
assert odd_count == 5, f"Expected 5 odd numbers, got {odd_count}"
assert even_count == 5, f"Expected 5 even numbers, got {even_count}"
assert odd_count + even_count == n, f"Total count should equal {n}"
print('Test passed: odd/even counting works correctly!')
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Generate Fibonacci Sequence

Exercise Description

Write a piece of code that generates the first n numbers of the Fibonacci sequence.

Requirements:

- Generate the first 7 Fibonacci numbers
- The Fibonacci sequence starts with 0 and 1, then each subsequent number is the sum of the two preceding ones
- Store all the generated numbers in a list called `fibonacci_numbers`
- For `n=7`, the sequence should be: [0, 1, 1, 2, 3, 5, 8]
- Return the variable `fibonacci_numbers`

Definition: Each number in the Fibonacci sequence is given by the sum of the two preceding ones. (The Fibonacci sequence starts with two 'seed' numbers 0 and 1) and then the proper sequence starts: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Your solution

```
def fibonacci(n):
    # step 1: start from an empty list
    # step 2: if n >= 1 append 0; if n >= 2 append 1
    # step 3: for i from 2 to n-1 compute next as sum of last two
    # step 4: append each new value
    # step 5: return the list
```

Test your solution

```
n = 7
fibonacci_numbers = fibonacci(n)
expected_sequence = [0, 1, 1, 2, 3, 5, 8]
assert fibonacci_numbers == expected_sequence, f"Expected {expected_sequence}, got {fibonacci_numbers}"
assert len(fibonacci_numbers) == n, f"Expected {n} numbers, got {len(fibonacci_numbers)}"
print('Test passed: Fibonacci sequence generation works correctly!')
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Multiples of 3 and 5

Exercise Description

Write a piece of code that processes numbers from 1 to n and creates a result list based on multiples.

Requirements:

- Process integers from 1 to n
- For multiples of both 3 and 5, add 'Mult35' to the result
- For multiples of 3 only, add 'Mult3' to the result
- For multiples of 5 only, add 'Mult5' to the result
- For other numbers, add the number itself to the result
- Store all results in a list called `result` and return it

Your solution

```
def classify_multiples(n):  
    # step 1: initialize empty result list  
    # step 2: for each num in 1..n check divisibility by 3 and 5  
    # step 3: append 'Mult35' if both, 'Mult3' if only 3, 'Mult5' if only 5, else num  
    # step 4: return result
```

Test your solution

```
n = 15  
result = classify_multiples(n)  
assert result[14] == 'Mult35', f"Expected 'Mult35' at position 14, got {result[14]}"  
assert result[2] == 'Mult3', f"Expected 'Mult3' at position 2, got {result[2]}"  
assert result[4] == 'Mult5', f"Expected 'Mult5' at position 4, got {result[4]}"  
assert len(result) == n, f"Expected {n} elements, got {len(result)}"  
print('Test passed: multiples classification works correctly!')
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Print Number Subset

Exercise Description

Write a piece of code that scans through a range of numbers but only collects a subset.

Requirements:

- Scan all integers from 1 to n
- Only collect numbers from 1 to m (where $m \leq n$)
- Store the collected numbers in a list called `subset_numbers` and return it

Your solution

```
def collect_subset(n, m):  
    # step 1: initialize empty subset list  
    # step 2: loop num from 1 to n inclusive  
    # step 3: if num <= m append to subset  
    # step 4: return subset
```

Test your solution

```
n = 10  
m = 6  
subset_numbers = collect_subset(n, m)  
expected_subset = [1, 2, 3, 4, 5, 6]  
assert subset_numbers == expected_subset, f"Expected {expected_subset}, got {subset_r  
assert len(subset_numbers) == m, f"Expected {m} numbers, got {len(subset_numbers)}"  
assert max(subset_numbers) == m, f"Expected max value {m}, got {max(subset_numbers)}"  
print('Test passed: subset collection works correctly!')
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Find Prime Numbers

Exercise Description

Write a piece of code that finds all prime numbers smaller than a given number n .

Requirements:

- Find all prime numbers smaller than n
- A prime number is greater than 1 and has no divisors other than 1 and itself
- Store all found prime numbers in a list called `prime_numbers` and return it
- Use a simple primality test: check if a number is divisible by any number from 2 to $n/2$

Your solution

```
def primes_less_than(n):  
    # step 1: initialize empty list of primes  
    # step 2: for each candidate from 2 to n-1 assume prime  
    # step 3: try divisors from 2 to candidate//2; if divisible mark not prime and break  
    # step 4: if still prime append to list  
    # step 5: return primes list
```

Test your solution

```
n = 15  
prime_numbers = primes_less_than(n)
```

```
expected_primes = [2, 3, 5, 7, 11, 13]
assert prime_numbers == expected_primes, f"Expected {expected_primes}, got {prime_numbers}"
assert all(p < n for p in prime_numbers), 'All primes should be less than n'
print('Test passed: prime number finding works correctly!')
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Find Pythagorean Triples

Exercise Description

Write a piece of code that finds all Pythagorean triples for a given number n .

Requirements:

- Find all pairs (x, y) such that $x^2 + y^2 = n^2$
- Both x and y should be positive integers less than n
- Store all valid pairs as tuples in a list called `pythagorean_pairs` and return it
- A Pythagorean triple consists of three positive integers x , y , and n such that $x^2 + y^2 = n^2$

Your solution

```
def pythagorean_pairs(n):  
    # step 1: initialize empty list  
    # step 2: for x in 1..n-1 and y in 1..n-1 compute x**2 + y**2  
    # step 3: if equals n**2 append (x, y)  
    # step 4: return the list
```

Test your solution

```
n = 5  
pythagorean_pairs_result = pythagorean_pairs(n)  
expected_pairs = [(3, 4), (4, 3)]  
assert pythagorean_pairs_result == expected_pairs, f"Expected {expected_pairs}, got {pythagorean_pairs_result}"  
for x, y in pythagorean_pairs_result:  
    assert x**2 + y**2 == n**2  
print('Test passed: Pythagorean triples finding works correctly!')
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

